



**CENTRO DE EDUCAÇÃO TECNOLÓGICA DO AMAZONAS
CETAM**

INSTITUTO BENJAMIN CONSTANT – IBC/CETAM

**PRÁTICA PROFISSIONAL SUPERVISIONADA III - PROJETO DE
SISTEMAS COMPUTACIONAIS WEB**

Ana Carolina Bomfim Bandeira

Breno Giovanni Batalha Frota

Marley Gabriel Alves Lemos

Wesley Neves Batista

Manaus

2026



Folha de Aprovação do Relatório de Atividades Práticas

Instituição onde foram realizadas as Atividades Práticas:

Endereço do local onde foram realizadas as Atividades Práticas:

—

Turno: _____

Período de realização: ____/____/____ a ____/____/____

Relatório de Atividades Práticas na
Etapa_____, apresentado ao
professor/orientador de Atividades Práticas,
como requisito parcial para aprovação no
componente
curricular_____,
parte integrante de Atividades Práticas
Profissional do Curso Técnico
em_____.

Nota Final das Atividades Práticas da Etapa ____: (____)

Considerações do Professor/Orientador de Atividades Práticas:

Professor/Orientador: _____

—



1. INTRODUÇÃO

Desenvolvemos o Sistema de Controle de Biblioteca como atividade da disciplina Prática Profissional Supervisionada III – Projeto de Sistemas Computacionais Web. Desde o início, nosso objetivo foi transformar os conteúdos estudados no curso em uma aplicação web funcional, capaz de representar situações reais de uma biblioteca, como cadastro de alunos e obras, organização do acervo, empréstimos, devoluções, multas, consultas e relatórios.

Durante o desenvolvimento, construímos o projeto de forma incremental. Começamos pela preparação do ambiente e pela modelagem do banco de dados e, à medida que as funcionalidades avançaram, estruturamos as camadas Model, DTO, Repository, Service, Controller e View. Utilizamos Java, Spring Boot, Spring Data JPA, MySQL, Thymeleaf e Bootstrap, além de Git e GitHub para o trabalho colaborativo. Essa evolução exigiu que voltássemos algumas vezes ao banco, às regras de negócio e à documentação para manter todas as partes coerentes entre si.

Na fase final, revisamos o banco de dados, acrescentamos o povoamento para facilitar os testes e a apresentação, reforçamos validações de unicidade e integridade, ajustamos os fluxos de empréstimo, renovação, devolução e multa e revisamos consultas, dashboard, relatórios e responsividade. Mais do que registrar funcionalidades prontas, este relatório apresenta como nós construímos o sistema, as decisões que tomamos, os problemas que encontramos, as correções realizadas e os conhecimentos que consolidamos ao longo do trabalho.



1.1 OBJETIVO GERAL

Projetar, implementar, integrar e documentar um Sistema de Controle de Biblioteca Web capaz de organizar o acervo e apoiar as rotinas de cadastro, consulta, empréstimo, devolução e multas, utilizando Java, Spring Boot, MySQL, Spring Data JPA, Thymeleaf e Bootstrap, com separação de responsabilidades entre as camadas da aplicação e persistência estruturada em banco de dados relacional.

1.2 OBJETIVOS ESPECÍFICOS

- Preparar e configurar o ambiente de desenvolvimento, banco de dados e ferramentas de versionamento utilizadas pela equipe.
- Modelar as entidades do domínio da biblioteca, seus atributos, chaves e relacionamentos, representando a estrutura tanto no banco de dados quanto nas classes Java.
- Implementar operações de cadastro, consulta, atualização e exclusão, respeitando validações e regras de integridade.
- Organizar o projeto em camadas Model, Repository, Service, Controller e View, reduzindo o acoplamento entre interface, regras de negócio e persistência.
- Implementar o controle de empréstimos, devoluções e multas com estados coerentes e vínculos entre aluno, livro e movimentação.
- Incluir o gerenciamento de categorias para melhorar a classificação dos livros e permitir que cada obra seja associada a uma categoria.
- Documentar as etapas realizadas e relacionar a prática desenvolvida aos conceitos técnicos estudados durante o curso.



2. DESENVOLVIMENTO

Nós desenvolvemos o projeto em etapas que foram se complementando. Primeiro organizamos o ambiente de trabalho e a base de dados; depois estruturamos as entidades e a persistência; em seguida implementamos regras de negócio, Controllers e interface; e, conforme testávamos os módulos em conjunto, voltávamos às etapas anteriores para corrigir relacionamentos, validações, rotas e comportamentos que só ficavam evidentes durante a integração. Esse processo foi importante para entendermos que o desenvolvimento de um sistema não acontece de forma totalmente linear.

2.1 Ferramentas utilizadas e preparação do ambiente

Na preparação do ambiente, buscamos manter uma base comum para que todos os integrantes conseguissem executar e alterar o projeto sem diferenças significativas de configuração. Utilizamos ferramentas para programação, banco de dados, modelagem, versionamento, documentação e organização das atividades. Ao longo do trabalho, percebemos que essa padronização foi essencial, principalmente quando alterações feitas por integrantes diferentes precisavam ser integradas no mesmo projeto.

Ferramenta/Tecnologia	Aplicação no projeto
Java	Linguagem principal do back-end. Foi utilizada para representar as entidades do domínio, implementar regras de negócio, validações e operações executadas pela aplicação.
Spring Boot	Framework central do projeto. Simplificou a inicialização da aplicação, configuração das dependências e criação dos componentes web e de persistência.
Spring Data JPA	Camada de abstração para acesso ao banco. Os Repositories estendem interfaces prontas, reduzindo a necessidade de escrever SQL para operações CRUD



	comuns.
Thymeleaf	Motor de templates utilizado na camada de apresentação para gerar páginas HTML com dados recebidos do back-end.
Bootstrap	Framework CSS empregado para construir uma interface organizada e responsiva, com componentes visuais reutilizáveis.
MySQL	SGBD relacional responsável pela persistência dos dados do sistema e pelo controle de chaves primárias, estrangeiras e restrições.
DBeaver	Ferramenta utilizada para acessar o MySQL, executar scripts, consultar registros e acompanhar a estrutura das tabelas.
MySQL Workbench	Utilizado na modelagem e representação visual do banco de dados e de seus relacionamentos.
IntelliJ IDEA	IDE utilizada na codificação do projeto Java/Spring, organização dos pacotes, execução e depuração.
Git e GitHub	Utilizados para controle de versão, compartilhamento do código e acompanhamento da evolução do projeto.
Trello	Apoio à organização das tarefas e acompanhamento das etapas do desenvolvimento em formato Kanban.
Draw.io	Utilizado na elaboração de diagramas e representações visuais utilizadas na documentação do sistema.
OpenHTMLtoPDF	Biblioteca utilizada pelo PdfReportService para converter os templates HTML processados pelo Thymeleaf em documentos PDF retornados diretamente ao navegador.

2.2 Levantamento de requisitos e documentação do sistema

Organizamos os requisitos no Documento de Requisitos de Software (DRS) e utilizamos esse documento como referência durante o desenvolvimento. Conforme o



projeto evoluiu, também revisamos o DRS, porque algumas decisões tomadas durante a implementação alteraram ou detalharam melhor o que havia sido definido inicialmente.

Entre as necessidades que trabalhamos estão o gerenciamento de livros, autores, editoras, categorias e alunos, além dos empréstimos, devoluções, multas, consultas e relatórios. Também consideramos aspectos de qualidade, como integridade dos dados, organização em camadas, manutenção do código e responsividade.

Um exemplo claro dessa evolução foi a inclusão de Categoria. Quando decidimos acrescentar essa entidade, percebemos que não bastava criar uma nova tabela: tivemos de alterar o relacionamento com Livro, criar Model, Repository, Service e Controller, ajustar formulários e atualizar a documentação. Esse tipo de mudança nos ajudou a compreender, na prática, como uma alteração aparentemente pequena pode afetar várias camadas do sistema.

2.3 Modelagem lógica e física do banco de dados

Modelamos o banco a partir das informações necessárias ao funcionamento da biblioteca e revisamos essa estrutura conforme novas regras surgiram. Na versão final, trabalhamos com autores, editoras, categorias, livros, alunos, empréstimos, multas e configurações do sistema. Definimos chaves primárias e estrangeiras para manter os vínculos entre os registros e utilizamos restrições para reduzir inconsistências.

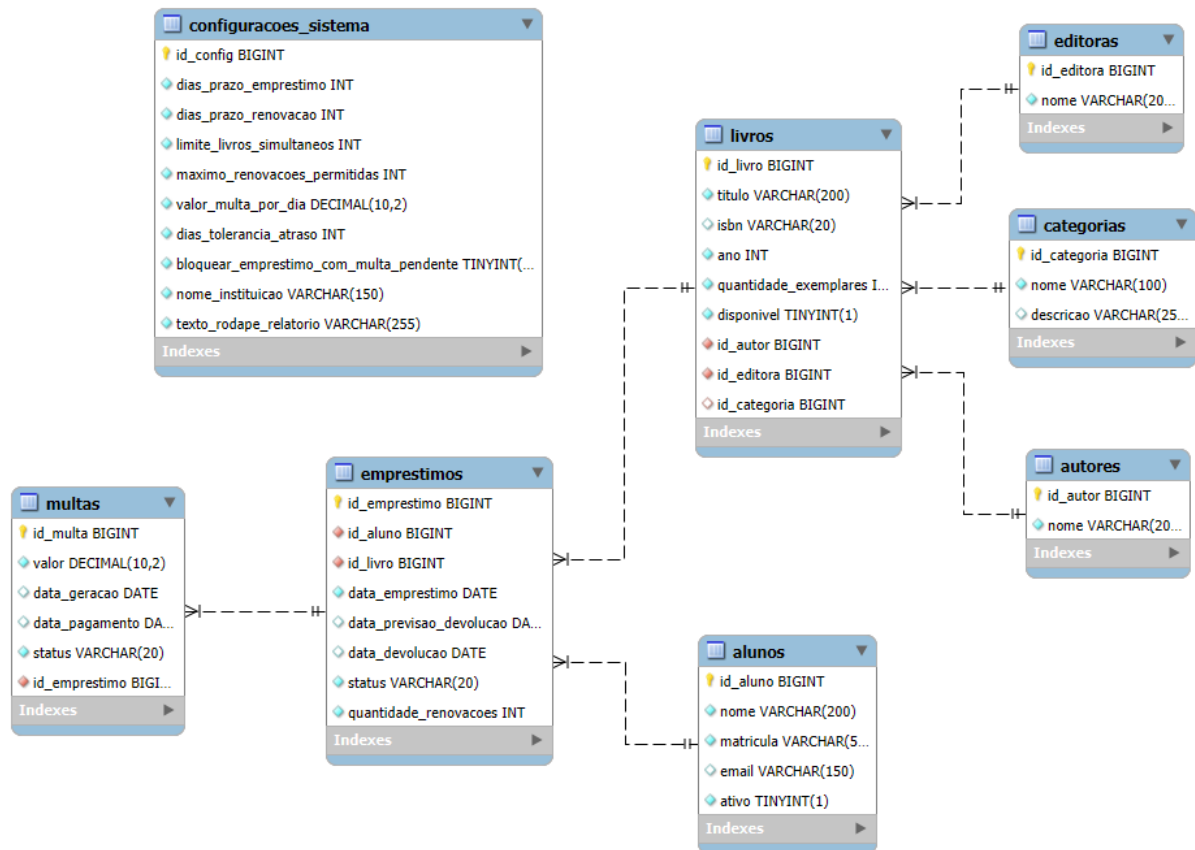


Figura 1 – Modelo lógico atualizado do banco de dados do Sistema de Controle de Biblioteca.

Tabela	Principais campos	Finalidade
autores	id_autor (PK), nome	Armazena os autores relacionados aos livros.
categorias	id_categoria (PK), nome, descricao	Classifica os livros do acervo e permite organizar as obras por categoria.
editoras	id_editora (PK), nome	Armazena as editoras responsáveis pelas publicações.
alunos	id_aluno (PK), nome, matricula, email, ativo	Representa os alunos aptos a utilizar os serviços da biblioteca.
livros	id_livro (PK), titulo, isbn, ano, quantidade_exemplares,	Representa as obras do acervo e mantém vínculos



	disponivel, id_autor, id_editora, id_categoria	com autor, editora e categoria.
emprestimos	id_emprestimo (PK), id_aluno, id_livro, datas e status	Registra a movimentação de um livro para um aluno e acompanha seu ciclo de devolução.
multas	id_multa (PK), valor, data_geracao, status, id_emprestimo	Registra penalidades associadas a um empréstimo específico.

O modelo lógico apresentado a seguir representa a estrutura que adotamos no projeto. Ele nos serviu como referência para conferir se os relacionamentos implementados nas entidades JPA permaneciam compatíveis com o banco MySQL.

2.4 Estruturação do projeto e arquitetura em camadas

Organizamos o código seguindo o padrão MVC e acrescentamos uma camada Service para concentrar as regras de negócio. Na prática, procuramos manter cada responsabilidade em seu lugar: o Controller recebe a requisição, o Service valida e executa a regra, o Repository acessa os dados e a View apresenta o resultado ao usuário. Os DTOs foram utilizados principalmente quando precisávamos transportar informações específicas para dashboard e relatórios.

Camada	Responsabilidade
Model	Representa as entidades do domínio e o mapeamento objeto-relacional com as tabelas do banco de dados.
DTO	Objetos de Transferência de Dados usados para consolidar indicadores e resultados de consultas, especialmente no dashboard e nos relatórios, sem expor



	diretamente as entidades JPA.
Repository	Responsável por operações de persistência, consultas e comunicação com o MySQL utilizando Spring Data JPA.
Service	Centraliza regras de negócio, validações e operações reutilizáveis antes do acesso aos Repositories.
Controller	Recebe requisições web, prepara dados para as páginas e coordena o fluxo entre interface e Service.
RestController	Disponibiliza endpoints HTTP em formato adequado para integração por API.
View	Camada de interface implementada com Thymeleaf, HTML, Bootstrap e CSS compartilhado. Contém formulários, listagens, dashboard, relatórios e fragmentos reutilizáveis.

Essa separação facilitou bastante as correções. Quando encontramos um problema em uma regra de empréstimo ou em uma validação, por exemplo, conseguimos atuar no Service sem misturar a correção com HTML ou acesso direto ao banco. Ao longo do projeto, essa organização tornou o código mais compreensível para todos nós.

2.5 Implementação das entidades e persistência com JPA

As classes do pacote Model representam as informações armazenadas no banco. Por meio das anotações do Jakarta Persistence/JPA, cada entidade é vinculada a uma tabela e seus atributos são associados às respectivas colunas. Chaves primárias são geradas automaticamente e os relacionamentos entre entidades são representados por associações orientadas a objetos.

Com a inclusão de Categoria, foi criada uma classe anotada como entidade e mapeada para a tabela categorias. Ela possui identificador do tipo Long, nome obrigatório com limite de 100 caracteres e descrição com limite de 255 caracteres. A classe também mantém uma coleção de livros associada pelo relacionamento um-para-muitos. Dessa forma, o modelo Java permanece alinhado ao modelo relacional.

A persistência da categoria é realizada por CategoriaRepository, que estende



JpaRepository<Categoria, Long>. Além das operações CRUD fornecidas automaticamente pelo Spring Data, foram definidos métodos para pesquisar categorias pelo nome de forma parcial e sem diferenciar maiúsculas de minúsculas, bem como verificar a existência de um nome já cadastrado. Esse recurso demonstra como o Spring Data JPA permite expressar consultas por meio da própria assinatura dos métodos.

2.6 Módulo de categorias: inclusão e integração com livros

Acrescentamos a entidade Categoria para melhorar a organização do acervo. Antes dessa evolução, os livros já estavam relacionados a Autor e Editora; com a nova entidade, passaram a possuir também uma classificação própria. Essa mudança facilita filtros, agrupamentos e futuras consultas por assunto ou tipo de obra, além de evitar o armazenamento repetido de descrições de categoria diretamente na tabela de livros.

No banco de dados, a tabela categorias possui id_categoria, nome e descricao. A tabela livros recebeu a chave estrangeira id_categoria. A restrição configurada permite que, em nível de banco, a referência do livro seja ajustada para nula caso a categoria seja removida, enquanto a camada de serviço do sistema adota uma regra mais restritiva e impede a exclusão de uma categoria quando existem livros vinculados. Essa validação na aplicação protege o usuário contra uma remoção que poderia deixar o acervo sem sua classificação planejada.

O CategoriaService concentra as operações de listagem, busca por identificador, salvamento e exclusão. A listagem aceita um parâmetro opcional de nome: quando o filtro é informado, a consulta procura ocorrências parciais ignorando diferenças entre maiúsculas e minúsculas; quando o filtro está vazio, todas as categorias são retornadas. Na exclusão, o Service consulta os livros associados e lança uma exceção quando encontra vínculos ativos, apresentando uma mensagem explicativa ao usuário.

Rota	Finalidade
GET /categorias	Lista categorias e permite filtro por nome na



	interface web.
GET /categorias/novo	Abre o formulário para cadastro de uma nova categoria.
GET /categorias/editar/{id}	Carrega uma categoria existente para edição.
POST /categorias/salvar	Valida o nome e persiste o cadastro ou alteração.
GET /categorias/excluir/{id}	Solicita exclusão e trata a existência de livros vinculados.
GET /api/categorias	Retorna todas as categorias pela API REST.
GET /api/categorias/{id}	Retorna uma categoria específica ou informa que não foi encontrada.
POST /api/categorias	Cria uma categoria via requisição HTTP.
PUT /api/categorias/{id}	Atualiza nome e descrição de uma categoria existente.
DELETE /api/categorias/{id}	Exclui a categoria quando as regras de integridade permitem.

A existência de um Controller tradicional e de um RestController para categorias oferece duas formas de acesso à mesma regra de negócio. O Controller atende às páginas renderizadas no servidor com Thymeleaf, enquanto o RestController expõe os dados por API. Como ambos utilizam CategoriaService, as regras centrais permanecem concentradas em um único ponto, evitando duplicação de lógica.

2.7 Gerenciamento de livros e integridade do acervo

O módulo de livros ocupa posição central no sistema porque se conecta a autores, editoras, categorias e empréstimos. Cada registro armazena título, ISBN, ano, quantidade de exemplares e indicador de disponibilidade, além das chaves que identificam os dados relacionados. Essa estrutura permite consultar informações bibliográficas e, ao mesmo tempo, controlar se a obra pode participar de uma nova operação de empréstimo.

A integridade referencial é importante principalmente no cadastro de livros.



Autor e Editora precisam existir para que suas chaves sejam vinculadas ao registro. Com a inclusão de Categoria, o formulário e as regras de persistência também precisam considerar essa associação. Essa abordagem impede o uso de referências inexistentes e mantém a consistência entre a camada de aplicação e o banco de dados.

2.8 Gerenciamento de alunos

A entidade Aluno representa os usuários que realizam empréstimos. O cadastro mantém identificador, nome, matrícula, e-mail e indicador de situação ativa. A matrícula possui característica de unicidade no banco, o que evita dois alunos com o mesmo código. O campo ativo permite inativar logicamente um aluno sem necessariamente remover seu histórico, solução importante quando já existem empréstimos ou registros relacionados.

Antes do registro de um empréstimo, o sistema deve verificar a existência do aluno e sua situação. Essa validação reduz a possibilidade de movimentações associadas a usuários inválidos ou inativos e reforça o uso das regras de negócio no Service em vez de depender apenas da interface.

2.9 Empréstimos e devoluções

A entidade Empréstimo registra a relação temporária entre um aluno e um livro. O registro contém a data do empréstimo, a previsão de devolução, a data efetiva de devolução e um status que representa a etapa atual da movimentação. Para criar um empréstimo, a aplicação precisa validar o aluno, identificar o livro e verificar sua disponibilidade antes de confirmar a operação.

Quando o empréstimo é efetivado, o sistema registra os dados da movimentação e atualiza a situação do livro. Na devolução, a data efetiva é registrada e o estado do empréstimo é alterado, permitindo devolver o livro ao fluxo de disponibilidade. A comparação entre a data prevista e a data efetiva também serve de base para identificar atrasos e permitir o tratamento de multas.

Os estados ATIVO, DEVOLVIDO e ATRASADO fornecem uma representação



clara do ciclo da movimentação. Esse controle facilita consultas posteriores, emissão de relatórios e identificação de pendências sem depender apenas das datas armazenadas.

2.10 Controle de multas e regras de negócio

Desenvolvemos o módulo de multas para registrar penalidades associadas a empréstimos. A entidade mantém valor, data de geração, status e referência ao empréstimo. Os estados PENDENTE e PAGO diferenciam valores ainda em aberto de pagamentos já concluídos. A relação com empréstimo mantém a rastreabilidade, pois permite identificar qual movimentação originou a cobrança e, indiretamente, qual aluno e livro estão relacionados.

Na camada Service, o sistema passou a contemplar consultas das multas cadastradas, pesquisa por aluno, separação entre pendentes e pagas, registro de pagamento, totais pendentes e recebidos e identificação de usuários com pendências. No pagamento, o status é atualizado para PAGO e a data correspondente é registrada. Também é necessário impedir que uma multa já liquidada seja paga novamente, preservando a consistência financeira do sistema.

A identificação do atraso é obtida pela comparação entre a data prevista para devolução e a data efetiva. Caso a devolução ainda não tenha ocorrido, a aplicação pode utilizar uma data de referência disponível para determinar o período em atraso. O projeto documenta a regra de cobrança por atraso e sua evolução deve permanecer sincronizada entre código, requisitos e testes para que o valor calculado represente a regra aprovada pela equipe.

2.11 Controllers, requisições HTTP e integração com a interface

Os Controllers atuam como pontos de entrada das solicitações feitas pelo navegador. Em uma operação de cadastro, por exemplo, o Controller recebe os campos enviados pelo formulário, verifica informações básicas, encaminha o objeto ao Service e, após a execução, direciona o usuário para a próxima página ou apresenta uma mensagem de sucesso ou erro. Esse fluxo evita o acesso direto da



interface ao banco de dados.

No módulo de categorias, o Controller utiliza caminhos específicos para listar, abrir formulário, editar, salvar e excluir. As mensagens são transmitidas para a interface por atributos do Model ou por RedirectAttributes. Quando uma categoria não pode ser removida por possuir livros associados, a exceção gerada na regra de negócio é transformada em uma mensagem compreensível para o usuário, melhorando a experiência e tornando a restrição explícita.

O RestController complementa essa arquitetura ao disponibilizar os dados por HTTP utilizando respostas com códigos adequados, como 200 para operações bem-sucedidas, 201 para criação, 404 para registro não encontrado e 409 para conflito de integridade. Isso prepara o sistema para integrações futuras com outros front-ends, aplicações móveis ou ferramentas de teste de API.

2.12 Interface web com Thymeleaf e Bootstrap

Planejamos a camada de apresentação para utilizar Thymeleaf e Bootstrap. O Thymeleaf permite que os dados fornecidos pelos Controllers sejam inseridos diretamente nas páginas HTML, possibilitando criar tabelas de listagem, formulários de cadastro e mensagens de retorno. O Bootstrap auxilia na organização visual e na responsividade, disponibilizando componentes como formulários, botões, tabelas, menus e grades adaptáveis a diferentes tamanhos de tela.

A separação entre Controller e View permite que a interface seja evoluída sem alterar as regras centrais de negócio. Assim, uma tela de categorias pode receber do Controller a coleção de registros já filtrada pelo Service e limitar-se a apresentá-los, mantendo a responsabilidade da busca na camada apropriada.

2.13 Controle de versão, organização da equipe e evolução incremental

Como o projeto foi desenvolvido em equipe, utilizamos Git e GitHub para registrar e compartilhar as alterações. Cada integrante trabalhou em partes do sistema e os commits nos ajudaram a acompanhar o que estava sendo incluído, corrigido ou ajustado. Também utilizamos o Trello como apoio para visualizar tarefas



e dividir atividades.

Um dos desafios do trabalho colaborativo foi manter alterações feitas em partes diferentes compatíveis entre si. Mudanças no banco, por exemplo, podiam exigir ajustes em Model, Repository, Service, Controller e View. Por isso, durante a integração, tivemos de revisar código, resolver diferenças entre versões e conferir se uma funcionalidade continuava funcionando depois da união das alterações.

Essa experiência tornou o uso de versionamento mais concreto para nós. O Git deixou de ser apenas uma ferramenta para salvar código e passou a fazer parte do processo de organização, revisão e integração do trabalho da equipe.

2.14 Validações, tratamento de erros e consistência

Durante os testes, percebemos que apenas fazer o cadastro ou a operação funcionar não era suficiente. Também precisávamos impedir estados inválidos e apresentar mensagens compreensíveis. Por isso, reforçamos validações para dados obrigatórios, duplicidade de matrícula, e-mail, ISBN e nomes, além de restrições relacionadas à exclusão de registros que já possuíam vínculos.

Esse foi um ponto importante do nosso aprendizado: várias falhas apareciam somente quando tentávamos repetir cadastros, excluir registros relacionados ou executar uma operação fora da sequência esperada. A partir desses cenários, ajustamos Services e Controllers para proteger a integridade do banco e melhorar o retorno apresentado ao usuário.

2.15 Testes e verificações realizadas durante o desenvolvimento

Nossos testes foram realizados principalmente durante a implementação e integração dos módulos. Executávamos a aplicação, acessávamos as telas, cadastrávamos dados, alterávamos registros e testávamos situações de empréstimo, devolução, multa, filtros e relatórios. Quando um erro aparecia, analisávamos em qual camada ele se originava e fazíamos a correção antes de continuar.

Entre os desafios encontrados estiveram erros de resolução e processamento de templates Thymeleaf, divergências entre rotas e nomes de arquivos, necessidade



de sincronizar alterações do banco com as entidades, comportamento da interface em larguras menores e regras que precisavam ser revistas depois que os módulos passaram a funcionar juntos. Essas correções fizeram parte do processo de aprendizagem e ajudaram a amadurecer a versão final.

Cenário de verificação	Resultado esperado
Cadastro de categoria válida	Registro persistido e mensagem de sucesso.
Categoria sem nome	Operação recusada com mensagem de campo obrigatório.
Filtro por nome	Retorno apenas das categorias correspondentes ao texto informado.
Edição de categoria	Dados atualizados sem alteração indevida do identificador.
Exclusão sem vínculos	Categoria removida normalmente.
Exclusão com livros vinculados	Operação bloqueada e mensagem explicativa ao usuário.
Busca por ID inexistente	Retorno de erro controlado, sem falha não tratada da aplicação.
Fluxo REST	Uso coerente de respostas HTTP para consulta, criação, atualização e exclusão.

2.16 Dashboard e módulo de relatórios

O desenvolvimento avançou para a estrutura responsável pela consolidação de dados do dashboard e do módulo de relatórios. Foram adicionados objetos DTO (Data Transfer Object), utilizados para transportar informações calculadas ou agrupadas entre as camadas da aplicação sem depender da exposição direta das entidades JPA.

O DashboardDTO reúne indicadores globais e dados operacionais, incluindo total de livros, exemplares e alunos; empréstimos ativos, atrasados e devolvidos; devoluções do dia; quantidade e valor de multas pendentes; taxa de circulação; últimos empréstimos; empréstimos críticos; livros mais emprestados e séries



destinadas à composição de gráficos. O `RelatorioGeralDTO` consolida totais gerais; o `ResumoFinanceiroDTO` organiza valores e quantidades de multas; o `LivroMaisEmprestadoDTO` representa o ranking de circulação; e o `CategoriaDistribuicaoDTO` representa a distribuição dos livros por categoria.

Em apoio a esses objetos, foram ampliados `Repositories` com operações de contagem, consultas por intervalo de datas e status, soma de valores de multas, totalização de exemplares, distribuição por categoria e levantamento dos livros mais emprestados. Essas consultas formam a base de dados necessária para a geração de indicadores operacionais e financeiros.

Também foram informadas pela equipe a implementação de `RelatorioService` e `RelatorioController`. O `Service` centraliza a composição das informações necessárias aos relatórios, mantendo as regras e agregações fora do `Controller`. O `Controller`, por sua vez, coordena as requisições web e disponibiliza os dados preparados à camada de apresentação. O `HomeController` recebeu ajustes relacionados à navegação da página inicial e do painel, mantendo a função de direcionamento das rotas principais.

Essa etapa representa uma evolução importante, relatórios deixam de ser apenas uma possibilidade futura e passam a possuir infraestrutura específica no back-end. A conclusão visual e os testes de integração do módulo permanecem como etapas de continuidade.

2.17 Implementação da camada View com Thymeleaf e Bootstrap

a interface web deixou de ser apenas uma etapa prevista e passou a integrar de forma efetiva o projeto. Foram criados templates para cadastro e listagem de alunos, autores, editoras, livros e empréstimos, além da listagem de multas, página inicial do dashboard e telas específicas de relatórios. Essas páginas utilizam atributos enviados pelos `Controllers` e expressões Thymeleaf para apresentar dados dinâmicos do banco de dados.

O Bootstrap foi utilizado em conjunto com uma folha de estilos própria para padronizar formulários, tabelas, cards, alertas, botões e comportamento responsivo. A integração exigiu também correções de rotas e nomes de templates para alinhar



os retornos dos Controllers à estrutura real de resources/templates.

2.18 Navegação reutilizável e organização do CSS

Para evitar duplicação do menu em cada página, foi criado o fragmento `fragments/sidebar.html`. O fragmento é incluído nos templates por Thymeleaf e centraliza os links de navegação para os módulos do sistema. A folha `static/css/sidebar.css` concentra o design compartilhado, incluindo estrutura da aplicação, sidebar, cards, tabelas, formulários, alertas e regras de adaptação para diferentes larguras de tela.

Essa organização melhora a manutenibilidade porque mudanças visuais ou de navegação podem ser realizadas em um ponto central, reduzindo divergências entre páginas e facilitando a evolução da interface.

2.19 Relatórios, filtros e geração de PDF

O módulo de relatórios foi ampliado com páginas para empréstimos, empréstimos atrasados, livros mais populares e demonstrativo financeiro de multas. Os relatórios aceitam filtros compatíveis com cada cenário, como período, status e limite de registros. Os dados são obtidos pelo `RelatorioService` a partir de consultas dos `Repositories` e `DTOs` específicos.

Para exportação, foi criado o `PdfReportService`. O serviço recebe o caminho de um template e um conjunto de dados, monta um `Context` do Thymeleaf, processa o HTML e utiliza `PdfRendererBuilder`, da biblioteca `OpenHTMLtoPDF`, para gerar o documento em memória. O `RelatorioController` retorna o conteúdo em `ResponseEntity` com `MediaType.APPLICATION_PDF`. Foram criados templates PDF específicos para empréstimos, atrasados, livros populares e financeiro.

Essa abordagem mantém a geração de PDF integrada ao mesmo modelo de dados utilizado nas páginas web, ao mesmo tempo em que separa a responsabilidade de conversão de documentos da lógica do Controller.

2.20 Integração entre os módulos do sistema



Na integração final, verificamos principalmente como os módulos dependiam uns dos outros. Um empréstimo, por exemplo, depende de um aluno válido, de um livro disponível, das configurações definidas e, em caso de atraso, pode gerar uma multa. Da mesma forma, alterações em autores, editoras e categorias influenciam o cadastro de livros.

Foi nessa etapa que conseguimos visualizar o projeto como um sistema único, e não como um conjunto de telas e classes separadas. Também foi quando identificamos ajustes que não eram evidentes enquanto cada módulo era testado isoladamente.

2.21 Módulo de configurações dinâmicas

Foi incorporado um módulo de configurações dinâmicas para retirar parâmetros operacionais do estado puramente fixo e permitir sua manutenção por uma tela administrativa. A entidade ConfigSistema é mapeada para a tabela `configuracoes_sistema` e mantém um registro principal de configuração.

Os parâmetros contemplam dias de prazo de empréstimo e renovação, limite de livros simultâneos, máximo de renovações permitidas, valor da multa por dia, dias de tolerância para atraso, opção de bloquear empréstimos quando houver multa pendente, nome da instituição e texto de rodapé dos relatórios. Os valores padrão incluem 14 dias para empréstimo, 14 dias para renovação, limite de 3 livros, até 2 renovações e multa diária de R\$ 2,00.

Distribuímos a implementação entre ConfigSistemaRepository, ConfigSistemaService e ConfiguracaoController. O Service utiliza @Transactional e valida os valores recebidos antes de persistir as alterações; também cria automaticamente a configuração padrão quando o registro ainda não existe e disponibiliza uma operação de restauração dos padrões. O Controller atende /configuracoes, prepara o objeto para a view, processa o salvamento e apresenta mensagens de sucesso ou erro por RedirectAttributes.

Também atualizamos o schema grupo03_db com a tabela `configuracoes_sistema` e uma carga inicial protegida por ON DUPLICATE KEY UPDATE, mantendo o registro padrão sem duplicação. Essa mudança aumenta a



manutenibilidade do sistema e permite que regras de circulação, cobrança e identificação institucional sejam ajustadas de forma centralizada.

2.22 Situação atual e possibilidades de evolução

Na versão que preparamos para a entrega, conseguimos integrar banco de dados, back-end e camada de apresentação nos principais módulos. Implementamos dashboard, relatórios com filtros e exportação em PDF, cadastros e listagens com Thymeleaf e Bootstrap, navegação compartilhada, configurações do sistema e ajustes de responsividade.

Mesmo com o escopo atendido, identificamos possibilidades de evolução que não fazem parte desta entrega, como autenticação e autorização por perfis, ampliação dos testes automatizados, auditoria de operações e melhorias adicionais para implantação em ambiente de produção. Registramos essas possibilidades como continuidade do aprendizado, e não como funcionalidades necessárias para considerar o projeto atual concluído.

2.23 Consolidação do banco de dados e atualização do schema

Na etapa final, o banco de dados foi revisado para manter correspondência entre o modelo físico, as entidades JPA e as regras implementadas. O schema utilizado é grupo03_db. O script principal cria autores, categorias, editoras, alunos, livros, empréstimos, multas e configuracoes_sistema, utilizando chaves primárias, chaves estrangeiras, restrições UNIQUE e CHECK de status para preservar a consistência dos dados.

A tabela livros referencia autor, editora e categoria; empréstimos referencia aluno e livro; multas possui vínculo único com o empréstimo; e configuracoes_sistema centraliza parâmetros operacionais. Essa estrutura fortalece a integridade referencial e reduz a necessidade de valores fixos distribuídos pelo código.



2.24 Povoamento do banco de dados

Na etapa final, acrescentamos o povoamento do banco para que o sistema pudesse ser demonstrado com uma quantidade representativa de dados já relacionados. Criamos registros de autores, categorias, editoras, alunos e livros e também cenários de empréstimos e multas. Isso facilitou nossos testes de listagens, filtros, atrasos, dashboard e relatórios, evitando que a apresentação dependesse de cadastros manuais feitos no momento da execução.

Também tivemos o cuidado de estruturar os inserts de modo coerente com os relacionamentos do banco, utilizando consultas para localizar registros por informações como matrícula, ISBN e nome. Com isso, o povoamento passou a ser parte importante da preparação do ambiente de demonstração.

2.25 Validações de unicidade, integridade e mensagens ao usuário

Reforçamos as validações na camada Service. Alunos são verificados quanto a dados obrigatórios e duplicidade de matrícula/e-mail; autores, editoras e categorias possuem verificações de nomes repetidos; livros exigem título, autor e editora, validam ISBN duplicado e repetição do mesmo título para o mesmo autor. A exclusão de livros é bloqueada quando existem empréstimos abertos ou histórico de movimentação.

Nos Controllers, exceções esperadas são transformadas em mensagens compreensíveis de erro ou sucesso. Essa estratégia mantém a regra de negócio no Service e deixa o Controller responsável pela resposta HTTP, redirecionamento e feedback ao usuário.

2.26 Regras consolidadas de empréstimo e renovação

O empréstimo verifica aluno existente e ativo, atrasos, eventual bloqueio por multa pendente, limite de livros simultâneos e disponibilidade de exemplar. Após a aprovação, registra datas/status, reduz a quantidade de exemplares e atualiza a



disponibilidade quando o estoque chega a zero. A renovação impede operações sobre empréstimos devolvidos ou vencidos e respeita prazo e quantidade máxima de renovações definidos em ConfigSistema.

2.27 Devolução, cálculo de multa e regularização do acervo

Na devolução, a aplicação registra a data efetiva, calcula o atraso em relação à previsão, desconta eventual tolerância e aplica o valor diário configurado. Quando há valor devido, cria multa PENDENTE vinculada ao empréstimo. O exemplar retorna ao estoque e a disponibilidade é atualizada.

O pagamento da multa registra a data de quitação e, caso o empréstimo relacionado ainda não esteja finalizado, conclui a devolução e repõe o exemplar. Esse ajuste evita divergências entre a situação financeira, o status do empréstimo e a disponibilidade do livro.

2.28 Otimização das consultas de atrasados e relatórios

Refinamos as consultas de atrasados para considerar a data prevista e o status operacional. O serviço também atualiza situações vencidas antes de determinadas listagens. RelatorioService utiliza DTOs e consultas agregadas para dashboard, empréstimos por período/status, atrasados, livros mais emprestados e resumo financeiro de multas. PdfReportService processa templates Thymeleaf e utiliza OpenHTMLtoPDF para exportação em PDF.

2.29 Configurações dinâmicas

ConfigSistema e ConfigSistemaService concentram prazo de empréstimo, prazo de renovação, limite de livros simultâneos, máximo de renovações, valor da multa por dia, tolerância de atraso, bloqueio por multa pendente, nome institucional e texto de rodapé dos relatórios. Essa solução facilita manutenção e permite adaptar regras sem espalhar constantes pelo código.



2.30 Verificação final da integração do sistema

Na nossa verificação final, percorremos os principais fluxos do sistema: cadastro, pesquisa, edição, exclusão protegida, empréstimo, renovação, devolução, multa, pagamento, filtros, dashboard e exportação de relatórios. Também revisamos o comportamento das telas em diferentes larguras para corrigir cortes e melhorar a apresentação.

Como o desenvolvimento foi dividido entre integrantes e branches, a integração do trabalho exigiu atenção especial. Em vez de tratar isso como uma observação externa sobre o repositório, consideramos essa etapa parte do próprio processo da equipe: conferimos as alterações produzidas, o povoamento e as correções finais para garantir que a versão entregue represente o conjunto do trabalho realizado.



3. CONCLUSÃO

Concluir este projeto foi importante para nós porque conseguimos aplicar, em uma única atividade, conhecimentos de Java, Spring Boot, banco de dados, JPA, desenvolvimento web, Git e documentação. Ao longo do trabalho, deixamos de enxergar as funcionalidades apenas de forma isolada e passamos a compreender melhor como banco, regras de negócio, Controllers e interface precisam permanecer sincronizados.

Os principais desafios surgiram justamente na integração. Tivemos de corrigir templates, rotas, relacionamentos, validações, regras de empréstimo e multa, consultas e detalhes de responsividade. Também enfrentamos o desafio de trabalhar em equipe com alterações acontecendo em partes diferentes do projeto. Resolver esses pontos exigiu comunicação, revisão do código e uso mais cuidadoso do controle de versão.

Outro aprendizado importante foi perceber que a documentação precisa acompanhar o sistema real. Sempre que alteramos o banco, acrescentamos Categoria, criamos configurações dinâmicas ou ajustamos uma regra de negócio, também foi necessário revisar o DRS e o relatório. Isso tornou a documentação parte do desenvolvimento, e não apenas uma etapa feita ao final.

Ao final, consideramos que o Sistema de Controle de Biblioteca atende ao objetivo proposto para a disciplina e representa de forma consistente o trabalho realizado pelo grupo. Além das funcionalidades implementadas, o projeto nos proporcionou experiência prática com organização em camadas, integração entre módulos, persistência de dados, tratamento de erros, trabalho colaborativo e revisão contínua de uma aplicação web.



4. REFERÊNCIAS

ORACLE. Java Documentation. Documentação oficial da plataforma Java. Acesso em: 27 ago. 2026.

SPRING. Spring Boot Reference Documentation. Documentação oficial do Spring Boot. Acesso em: 27 ago. 2026.

SPRING. Spring Data JPA Reference Documentation. Documentação oficial do Spring Data JPA. Acesso em: 27 ago. 2026.

THYMELEAF. Thymeleaf Documentation. Documentação oficial do mecanismo de templates. Acesso em: 27 ago. 2026.

MYSQL. MySQL 8.0 Reference Manual. Documentação oficial do MySQL. Acesso em: 27 ago. 2026.

BOOTSTRAP. Bootstrap Documentation. Documentação oficial do framework de interface. Acesso em: 27 ago. 2026.

GIT. Git Documentation. Documentação oficial do sistema de controle de versão. Acesso em: 27 ago. 2026.